

Kunjika

A low-latency voice activator: keyword spotting on an ESP32-S3, streaming to a speech-recognition server

Team documentation: what we built, how every part works, why we made each decision, what we measured, and what every important term means. Written so that any teammate can understand, run, explain and defend the whole project.

Wake word	Kunjika (Sanskrit for 'key'): chosen because it is rare in everyday speech
Edge device	ESP32-S3-DevKitC-1 (N16R8) + INMP441 digital microphone
On the device	int8 DS-CNN, 16.8 KB, TensorFlow Lite Micro, 6 ms per run
Server	Python + faster-whisper (open-source Whisper), Hinglish output
Status	complete end-to-end system, measured against every metric of the problem statement

Contents

1. The problem we are solving	5
1.1 How the solution will be judged	5
1.2 The hard rules	5
1.3 Why ISRO cares	5
1.4 Our keyword: Kunjika	5
2. What we built: the big picture	7
2.1 The headline numbers	7
3. One command, step by step	9
4. The hardware	10
4.1 The ESP32-S3 board	10
4.2 The INMP441 microphone	10
4.3 The LED	11
5. Sound as numbers: the basics	12
5.1 Sample rate: 16,000 per second	12
5.2 Bits and loudness: dBFS	12
5.3 I2S: how the microphone talks	12
5.4 Frames and hops: the 20 ms heartbeat	12
6. From sound to features: the log-mel spectrogram	13
6.1 The steps	13
6.2 Bit-exact with TensorFlow	13
6.3 The 60 Hz floor and the features file	14
6.4 Cost	14
7. The neural network: DS-CNN	15
7.1 What a neural network is	15
7.2 Convolution, in one paragraph	15
7.3 Depthwise separable convolution: why “DS”	15
7.4 Our architecture, layer by layer	15
7.5 Making it tiny: int8 quantization and QAT	16
7.6 Running it on the board: TensorFlow Lite Micro and ESP-NN	16
8. Deciding “was that Kunjika?”	18
8.1 A sliding window	18
8.2 The four rules	18
8.3 The trade-off	18
9. Data: what the model learned from	19
9.1 The recording protocol	19
9.2 What we collected	19
9.3 Preparing the data	20
9.4 Splitting by speaker	20
9.5 Synthetic voices (TTS)	20
10. Training the model	21
10.1 Augmentation: making the data harder	21
10.2 What goes into each training batch	21
10.3 How we measure accuracy	21
10.4 Evaluating exactly like the board	22
10.5 Three versions and what each taught us	22
11. The firmware on the ESP32-S3	23
11.1 Tasks and cores	23
11.2 The listening path (core 1)	23
11.3 The streaming path (core 0)	24
11.4 Networking	24
11.5 Reading the stats line	25
11.6 The microphone health check	25
11.7 The memory diet: from ~383 KB to ~243 KB	25
12. How the board and the server talk	27

12.1 Clock synchronisation: measuring latency honestly	27
13. The server: speech recognition	28
13.1 What it does for every session	28
13.2 Whisper	28
13.3 Only the command, never the keyword	28
13.4 English, Hindi and Hinglish	28
13.5 Guards against Whisper's bad habits	29
13.6 Keeping the network side fast	29
13.7 Optional: a Hindi specialist model	29
14. Results: what we measured	30
15. Bugs we hit, and what they taught us	31
16. What is still weak, and the plan	32
17. Running everything yourself	33
17.1 The server (laptop)	33
17.2 The board	33
17.3 When something goes wrong	33
17.4 Where everything lives	33
18. Glossary	35

How to read this document

This document explains the whole Kunjika project from the ground up. It is written for teammates who did not build every part themselves, and for anyone who has to explain the project to judges. Nothing is assumed except school-level maths and some curiosity.

Chapters 1 to 3 give the big picture: the problem, what we built, and what happens, step by step, when someone says *“Kunjika, turn on the lights”*. If you only have ten minutes, read these.

Chapters 4 to 13 go through every part in depth: the hardware, how sound becomes numbers, the features, the neural network, the decision logic, the data, the training, the firmware on the board, the network protocol and the server.

Chapters 14 to 17 cover the measured results, the bugs we hit and what they taught us, what is still weak and how we plan to fix it, and how to run everything yourself.

Chapter 18 is a glossary. Whenever a technical word appears in **bold** and you are not sure what it means, look it up there. Every term is explained in plain language.

Two kinds of boxes appear throughout. An orange **“In simple words”** box restates a hard idea without jargon. A green **“Why we did it this way”** box explains a design decision: what the alternatives were, and why we chose what we chose. Judges often ask “why?”, so those boxes matter.

1. The problem we are solving

The Indian Space Research Organisation (ISRO) posted problem statement 26172 for Smart India Hackathon 2026. Its background, in its own spirit: voice-controlled devices are everywhere, but sending *everything* a microphone hears to the cloud is expensive, slow and bad for privacy. The future is a **hybrid architecture**: a small, cheap device listens locally for one special word, and only after hearing it does it wake up the powerful cloud to understand the rest of the sentence.

So the task has two halves. First, build an **ultra-lightweight keyword spotting (KWS)** model that runs on a low-power microcontroller and listens all the time. Second, when the keyword is heard, **stream the audio that follows** to a remote automatic speech recognition (**ASR**) server, quickly and with as little data as possible.

1.1 How the solution will be judged

Metric	What it means	Where we stand
Efficiency	Model size (RAM and flash) and CPU usage while idly listening	16.8 KB model; ~243 KB RAM for the whole firmware incl. Wi-Fi; 9.4% of the detection core
Accuracy	High true-positive rate for the keyword, near-zero false activations	88% on normal speech from unseen speakers; 0 false activations in 18.8 min of podcast; conversation still being improved
Latency	Time from the keyword ending to the ASR server receiving the audio	median ~170 ms, of which the network is ~13 ms

1.2 The hard rules

- The edge software must run within **less than 256 KB of RAM**.
- It must use **under 10% CPU** while idly listening.
- **Open source only**. No proprietary or commercial voice-activation SDKs.
- **No pre-trained global keywords** such as “Hey Google” or “Alexa”: we must train on our own custom keyword.
- It will be evaluated on a **physical low-power microcontroller** such as an ESP32 or a Raspberry Pi. Heavy, uncompressed transformer models are disqualified.

1.3 Why ISRO cares

ISRO wants a stack that is **indigenous and auditable**: every line of code can be inspected, no black-box foreign SDK sits in the middle. Think of technicians in a cleanroom wearing gloves, or a ground station running on limited power and bandwidth: they need hands-free control that is fast, cheap to run, and does not send every word to someone else's server. Everything we used is open source: TensorFlow, TensorFlow Lite Micro, ESP-IDF, ESP-NN, esp-dsp, Whisper and Piper.

1.4 Our keyword: Kunjika

Kunjika is Sanskrit for “key”. A wake word should be **rare in normal speech** (so the device does not wake up by accident), **easy to say**, and **long enough** to be distinctive. Three

syllables, ku-nji-ka, is a good length. Its main risk, which we discovered and measured, is that some common Indian names end with a similar sound (Kritika, Ankita, Ambika). Chapter 10 explains how we are dealing with that.

In simple words

We built a tiny “ear” that sits on a small chip and does only one thing: it waits to hear “Kunjika”. The moment it does, it passes whatever you say next to a big computer that writes it down. The tiny ear must be very frugal (little memory, little CPU), and the handover must be very fast.

2. What we built: the big picture

The system has three physical parts: a digital **microphone**, the **ESP32-S3 board** (the “edge”), and a **server** (a laptop standing in for “the cloud”). The board's processor has two cores, and we give each one a clear job.

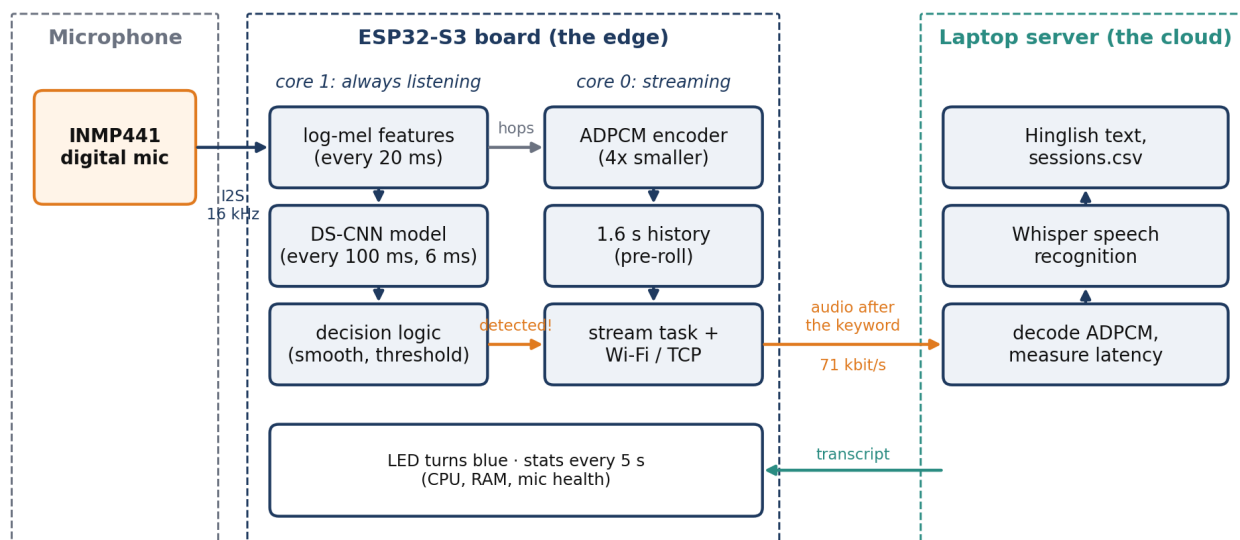


Figure 1. The whole system. Core 1 of the board does nothing but listen for the keyword; core 0 compresses audio and handles the network; the server turns speech into text.

Core 1 runs the always-on listening pipeline. Every 20 milliseconds it takes the newest slice of sound, turns it into a compact description called **log-mel features**, and every 100 ms it asks a small neural network, **DS-CNN**, “is this Kunjika?”. A little decision logic smooths the answers so a single noisy guess cannot trigger the device.

Core 0 quietly compresses every slice of audio with **ADPCM** (four times smaller) and keeps the last 1.6 seconds in memory. When core 1 says “detected!”, core 0 opens a **session**: it sends the last second (which contains the keyword and the start of the command, the **pre-roll**) in one burst, then keeps sending live audio until the speaker stops talking. The network connection to the server is kept open all the time, so no time is wasted connecting.

The **server** receives the audio, measures exactly how long it took to arrive, and hands it to **Whisper**, an open-source speech-recognition model. English is written as English; Hindi is recognised as Hindi and written in Hinglish (Roman letters). The text goes back to the board, and every session is logged in a spreadsheet file for statistics.

2.1 The headline numbers

What	Measured	Requirement
Keyword end → server receiving audio	median ~170 ms, 90th percentile ~255 ms (12 sessions)	measured as defined
...of which network	median 13 ms	
CPU while idle listening	9.3–9.6% on the detection core; 6.3% chip average	< 10%

What	Measured	Requirement
RAM, complete firmware incl. Wi-Fi	~243 KB at peak (137.7 static + 105 heap); ~227 KB idle	< 256 KB
Model	16.8 KB int8 file, 16.9 KB working memory, 6 ms per run	ultra-lightweight
Audio on the network	71 kbit/s instead of 256 kbit/s raw	minimal overhead
Keyword accuracy (model v3)	88% on normal speech, unseen speakers; 0 false activations in 18.8 min podcast	high hit rate, near-zero false activations
Reliability	0 dropped audio; recovers from Wi-Fi or server loss by itself	

How we get these numbers

Every number in this document was measured on the real hardware or computed from real recordings; none is an estimate unless we say so. The board prints its own CPU, RAM and audio statistics every 5 seconds, and the server logs every session's latency to *sessions.csv*. Chapter 14 explains how each measurement is taken.

3. One command, step by step

Let us follow a single command through the whole system: somebody says “*Kunjika, turn on the lights*”. The times are typical values from our measurements.

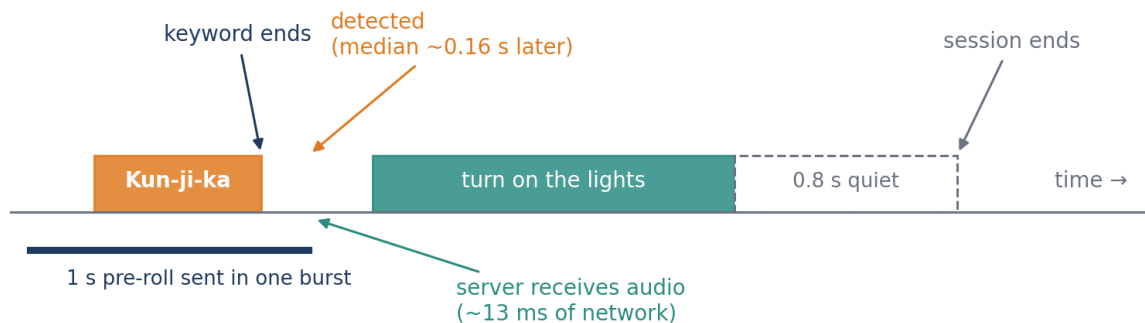


Figure 2. Timeline of one command. The latency the problem statement asks about is the gap between “keyword ends” and “server receives audio”.

1. **Sound becomes numbers.** The INMP441 microphone converts air pressure into 16,000 numbers per second and sends them to the ESP32-S3 over three wires (**I2S**).
2. **Every 20 ms, a new column of features.** Core 1 takes the newest 20 ms of sound and computes 40 numbers describing how much energy there is in each pitch band. The last 49 columns form a 1-second “picture” of sound.
3. **Every 100 ms, the model looks.** The DS-CNN network looks at that picture and outputs a probability that it contains “Kunjika”. This takes about 6 ms.
4. **The decision.** The average of the last three probabilities crosses 0.80, typically 0.1 to 0.3 s after the word ends. The board prints *DETECTED* and the LED turns blue.
5. **The handover.** Core 0 immediately sends the last 1 second of compressed audio in one burst, then the live audio, 20 ms at a time. The server receives the first audio about 13 ms later.
6. **End of the command.** When the speaker has been quiet for 0.8 s, the board ends the session.
7. **Transcription.** The server cuts off the keyword, sends only “turn on the lights” to Whisper, and a few seconds later sends back the text: *Kunjika, turn on the lights*.

In simple words

From the moment you finish saying “Kunjika”, the server is receiving your voice in about a sixth of a second. Writing the text down takes a few more seconds, but only because Whisper runs on a laptop’s CPU; a real cloud server with a GPU would do it in a fraction of a second. That writing-down time is not part of the latency the problem statement measures.

4. The hardware

4.1 The ESP32-S3 board

The **ESP32-S3** is a widely used, low-cost microcontroller made by Espressif Systems. Our board is the **ESP32-S3-DevKitC-1, variant N16R8**: 16 MB of flash storage and 8 MB of extra PSRAM. Everything that runs on it (ESP-IDF, FreeRTOS, TensorFlow Lite Micro, ESP-NN, esp-dsp and our code) is open source.

Feature	Value	Why it matters to us
Processor	2 cores (Xtensa LX7), 240 MHz	one core listens, the other streams: they never get in each other's way
Vector instructions	yes (SIMD)	ESP-NN uses them to run the neural network several times faster
Internal RAM (SRAM)	512 KB	everything we run lives here; the 256 KB limit is measured against it
PSRAM	8 MB (deliberately NOT used)	using it would make the 256 KB claim meaningless, so it is switched off
Flash	16 MB	stores the program and the model (the model is read straight from flash)
Wi-Fi	2.4 GHz 802.11 b/g/n	carries the audio to the server; 5 GHz networks are invisible to it

Why we did it this way

We needed Wi-Fi (for streaming), enough speed for a neural network, and vector instructions. The ESP32-S3 has all three for a few hundred rupees. The problem statement names the ESP32 family explicitly. We switched the 8 MB PSRAM **off**, so every byte we use comes from the chip's own internal RAM, which is what makes our “under 256 KB” claim honest.

4.2 The INMP441 microphone

The **INMP441** is a tiny **MEMS** microphone (a microscopic vibrating membrane on a silicon chip) with a built-in analog-to-digital converter. It sends **digital** numbers directly, so there is no analogue wire to pick up electrical noise. It talks the **I2S** protocol, and we wire it like this:

INMP441 pin	ESP32-S3 pin	Meaning
VDD	3V3	power (3.3 volts)
GND	GND	ground
SCK	GPIO 4	bit clock: one tick per bit sent
WS	GPIO 5	word select: says whether the left or right channel is being sent
SD	GPIO 6	serial data: the audio bits themselves
L/R	GND	tells this microphone to be the LEFT channel

Lesson learned: solder the pins

The INMP441 module ships with its header pins loose. For weeks we pushed the pins through the holes without soldering, and the microphone failed on and off: sometimes exact zeros (“digital silence”), sometimes garbage bit patterns (0, 1, 3, 7, ... 511) from a floating data wire. Whisper then “heard” words such as *you* in silence, and we lost days thinking the software was wrong. Soldering the pins fixed it permanently. The firmware now detects both failure patterns and prints *MIC FAULT* (section 11.6).

4.3 The LED

The board's RGB LED (GPIO 38 on the v1.1 board, GPIO 48 on v1.0) blinks green at start-up and turns **blue** when the keyword is detected. It is useful in the demo video, because it shows the detection instantly, without anyone reading a log.

5. Sound as numbers: the basics

Sound is a wave of air pressure. The microphone's membrane moves with the pressure, and the converter measures its position many times per second. Each measurement is a **sample**: one number.

5.1 Sample rate: 16,000 per second

We take **16,000 samples per second** (16 kHz). A rule called the **Nyquist limit** says that with 16,000 samples per second you can capture frequencies up to half of that, 8,000 Hz. Human speech carries almost all of its meaning below 8 kHz, which is why telephones and nearly all speech recognisers use 16 kHz. Higher rates would only cost more memory and CPU.

We measured the real rate on our board at **16,014 Hz**, only 0.09% off, and over 60 seconds of recording not a single sample was lost.

5.2 Bits and loudness: dBFS

Each sample is stored as a **16-bit** integer: a whole number between $-32,768$ and $+32,767$. Silence is near 0; loud sounds swing towards the extremes. The INMP441 actually delivers 24 bits inside a 32-bit slot; we keep the top part by shifting right by 16 bits ($GAIN_SHIFT = 16$), a setting we measured and then froze.

Loudness is reported in **dBFS** (decibels relative to full scale). 0 dBFS is the loudest possible signal; every -6 dB halves the amplitude. On our board, a quiet room reads about **-35 to -45 dBFS**, speech about -15 to -25 dBFS, and **-120 dBFS** is the value we print for exact digital silence: a sure sign the microphone is disconnected.

5.3 I2S: how the microphone talks

I2S (Inter-IC Sound) is a standard three-wire way to send audio between chips. The clock wire ticks once per bit, the word-select wire flips between left and right channel, and the data wire carries the bits. At 16 kHz with two 32-bit channels, the clock ticks about 1 million times per second. The ESP32's I2S hardware uses **DMA** (direct memory access): it fills a memory buffer by itself, without the CPU, and only wakes our code when a block of samples is ready.

5.4 Frames and hops: the 20 ms heartbeat

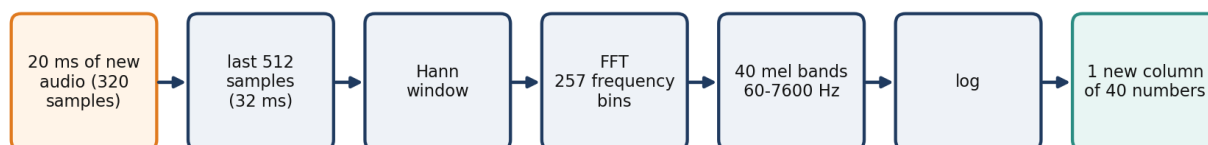
Everything in our system ticks every **20 milliseconds**. That period is called a **hop**, and one hop is 320 samples. Each hop, the board computes one new column of features, hands the raw audio to the encoder, and updates its statistics. A **frame** is the slightly longer window of 512 samples (32 ms) that the features are computed from, so consecutive frames overlap.

In simple words

The microphone produces 16,000 numbers a second. We work on them in small chunks of 320 numbers (20 ms). Loudness is measured in dBFS, where 0 is the maximum and lower numbers are quieter; -120 means “nothing at all is coming from the microphone”.

6. From sound to features: the log-mel spectrogram

A neural network could, in principle, look at the raw samples. In practice that wastes effort: raw samples are 16,000 numbers per second, and most of that detail (the exact phase of each wave) says nothing about *which word* was spoken. What matters is **how much energy there is at each pitch, over time**. So we convert sound into a picture of exactly that, called a **log-mel spectrogram**.



49 columns side by side = 1 second of sound = the 49 × 40 'picture' the model looks at

Figure 3. How one column of features is made, every 20 ms.

6.1 The steps

1. **Take the last 512 samples** (32 ms). Speech sounds change quickly, but within about 30 ms a sound is roughly steady, so that is the right size to analyse.
2. **Apply a Hann window**. We gently fade the 512 samples in and out at the edges. Without this, the sudden cut at the edges would create fake high frequencies.
3. **FFT (Fast Fourier Transform)**. This splits the 512 samples into 257 **frequency bins**: how much energy there is at 0 Hz, 31.25 Hz, 62.5 Hz ... up to 8,000 Hz. We square the values to get **power**.
4. **Mel filterbank**. We group the 257 bins into **40 mel bands** between 60 and 7,600 Hz. The bands are narrow at low pitches and wide at high pitches, because human hearing works that way (the **mel scale**). This cuts the data six-fold while keeping what matters for speech.
5. **Logarithm**. We take the log of each band's energy (plus 10^{-6} so silence does not give log of zero). Our ears perceive loudness logarithmically, and it keeps loud and quiet speech on a similar scale.
6. **Stack 49 columns**. The newest 49 columns, one per hop, cover about one second. This 49 × 40 grid of numbers is the “picture” the neural network looks at.

6.2 Bit-exact with TensorFlow

The model was trained in **TensorFlow** on a laptop, but it runs on the ESP32 with our own C code for the features. If the board computed features even slightly differently from TensorFlow, the model would be seeing a picture it was never trained on, and accuracy would quietly collapse. So we tested it: we fed the same audio to both and compared all **1,960 numbers** (49 × 40) of the input. **All 1,960 were identical**. We also export TensorFlow's exact mel filter weights into the firmware rather than recomputing them.

A subtle trap we found

There are two versions of the Hann window: “periodic” (dividing by N) and “symmetric” (dividing by $N - 1$). TensorFlow uses the periodic one. Using the other version made our features almost, but not exactly, right. This kind of invisible mismatch is exactly why we insisted on a bit-exact comparison.

6.3 The 60 Hz floor and the features file

The INMP441 picks up a lot of very low-frequency rumble: breathing, air movement, handling noise. In silence recordings, **63 to 87% of all energy was below 20 Hz**. For the Kunjika models we start the mel bands at **60 Hz** instead of 20 Hz, so that rumble never enters the features. Because the model and the firmware must agree on this, training writes a small file, *model.features.json*, and the header generator reads it and refuses to build a mismatched firmware. We tested what a mismatch looks like: with the wrong setting, only 367 of the 1,960 input numbers matched.

6.4 Cost

One column takes about **333 microseconds** to compute, done 50 times a second: about **1.7%** of one CPU core. Cheap.

In simple words

We turn sound into a picture: time goes left to right, pitch goes bottom to top, and brightness shows how loud each pitch is. The neural network looks at one second of this picture at a time. We made sure the board draws exactly the same picture, number for number, as the software the model was trained with.

7. The neural network: DS-CNN

7.1 What a neural network is

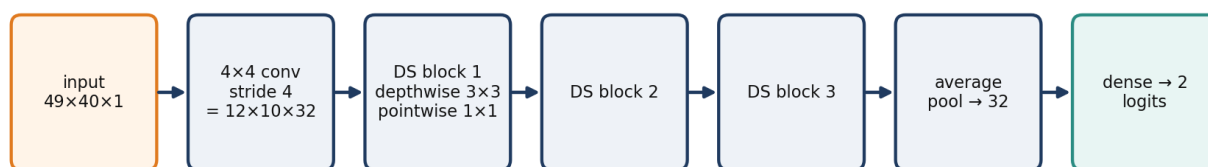
A **neural network** is a mathematical function with many adjustable numbers, called **weights**. It takes an input (our 49×40 picture) and produces an output (two numbers: one for “Kunjika”, one for “not Kunjika”). **Training** means showing it thousands of labelled examples and nudging the weights, a tiny bit each time, so that its outputs get closer to the right answers. After training, the weights are frozen and the network is used for **inference**: making predictions on new audio.

7.2 Convolution, in one paragraph

A **convolution** slides a small pattern detector (for example 3×3 numbers) across the picture and measures, at every position, how well that spot matches the pattern. One detector might respond to a rising pitch, another to a burst of noise such as the “k” in Kunjika. A layer has many detectors, called **channels**; ours has 32. Stacking layers lets later layers combine simple patterns into complex ones, such as “this sequence of sounds is ku-nji-ka”.

7.3 Depthwise separable convolution: why “DS”

A normal 3×3 convolution from 32 channels to 32 channels needs $3 \times 3 \times 32 \times 32 = 9,216$ multiply-adds at every position. A **depthwise separable** convolution splits this into two cheap steps: a **depthwise** 3×3 filter applied to each channel on its own ($3 \times 3 \times 32 = 288$), then a **pointwise** 1×1 convolution that mixes the channels ($32 \times 32 = 1,024$). The total is **1,312**, about 7 times cheaper, with nearly the same accuracy. **DS-CNN** (depthwise separable convolutional neural network) is a standard, well-tested design for keyword spotting on microcontrollers.



16.8 KB after int8 quantization · about 0.55 million multiply-adds · 6 ms per run on the ESP32-S3

Figure 4. Our DS-CNN, layer by layer, with the size of the data after each step.

7.4 Our architecture, layer by layer

Layer	Output shape	What it does
Input	$49 \times 40 \times 1$	one second of log-mel features
Conv 4x4, stride 4	$12 \times 10 \times 32$	looks at 4×4 patches, jumping 4 at a time: shrinks the picture 16-fold at once
3 × DS block	$12 \times 10 \times 32$	each: depthwise $3 \times 3 \rightarrow$ batch norm $\rightarrow$ ReLU $\rightarrow$ pointwise $1 \times 1 \rightarrow$ batch norm $\rightarrow$ ReLU

Layer	Output shape	What it does
Average pool	1 × 1 × 32	averages each channel over the whole picture: “how much of pattern k is present?”
Dropout 0.2	32	training only: randomly hides 20% of values so the model cannot over-rely on any one
Dense	2	two final scores (logits): “Kunjika” and “not Kunjika”

Batch normalisation keeps the numbers inside the network in a steady range, which makes training stable. **ReLU** simply replaces negative numbers with zero; that small bend is what lets a network learn complicated shapes instead of just straight lines.

The two outputs are **logits**: raw scores, not probabilities. On the board we turn them into a probability with the **sigmoid** of their difference: $p = 1 / (1 + e^{-(\text{pos} - \text{neg})})$. This is mathematically the same as the usual softmax for two classes, but cheaper (section 7.6).

7.5 Making it tiny: int8 quantization and QAT

Normally a network's weights are 32-bit floating-point numbers (**float32**). **Quantization** stores each one as an 8-bit integer (**int8**, from -128 to 127), plus a **scale** and a **zero point** that say how to convert back: $\text{real value} \approx \text{scale} \times (\text{integer} - \text{zero point})$. The model becomes **4 times smaller**, and integer maths is much faster on a microcontroller. Our final model file is **16,808 bytes**. For example, the input uses scale 0.0944 and zero point 18.

Rounding to 8 bits loses a little precision. To avoid losing accuracy, we use **quantization-aware training (QAT)**: during the last training epochs, the network is made to “feel” the rounding, so it learns weights that still work after quantization.

7.6 Running it on the board: TensorFlow Lite Micro and ESP-NN

TensorFlow Lite Micro (TFLM) is Google's open-source runtime for neural networks on microcontrollers. It reads the model file and executes it layer by layer. It needs a block of working memory called the **tensor arena**, which holds the data flowing between layers; ours uses **16,892 bytes** of an 18 KB arena. **ESP-NN** is Espressif's open-source library of hand-optimised layer implementations that use the ESP32-S3's vector instructions.

We profiled the model **layer by layer** on the chip, and three measurements changed the design:

- **The first layer was the bottleneck.** ESP-NN's fast path needs at least 16 input channels, and the first layer has only one (a single feature picture), so it always ran on the slow general path. Its cost grows with the number of *outputs* it produces. Replacing a 3×3 stride-2 first layer (16,000 outputs) with a **4×4 stride-4** “patch” layer (3,840 outputs) cut it from about 10 ms to about 2 ms.
- **Averaging was slow the “wrong” way.** TFLM's MEAN operation took **3.1 ms** to average 3,840 numbers; AVERAGE_POOL_2D does the same maths in **60 μ s**. We switched.
- **Softmax was unnecessary.** Converting two scores to probabilities with softmax cost 453 μ s. We output raw logits and compute one sigmoid ourselves.

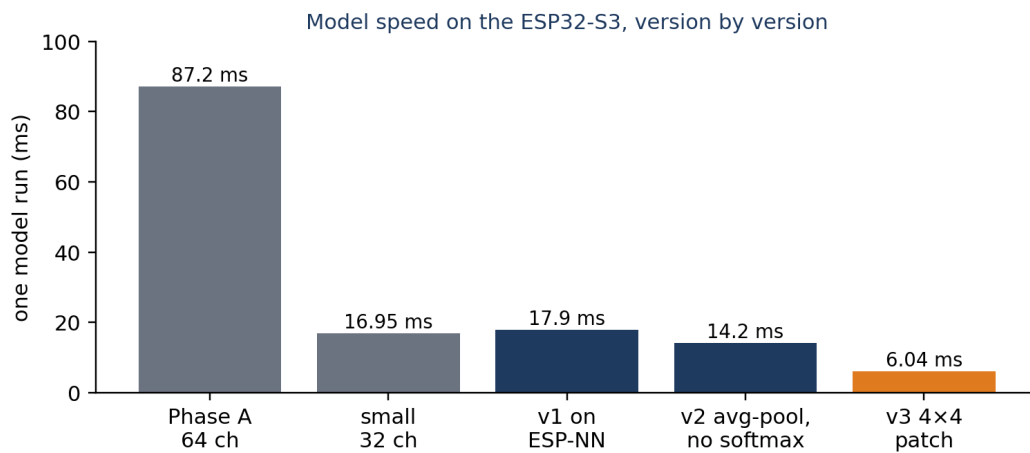


Figure 5. One model run on the ESP32-S3: from 87 ms to 6 ms through measurement-driven design. (The first two bars used slower reference kernels.)

Why we did it this way

We did not guess where the time went: we measured every layer on the real chip, then changed exactly the parts that were slow. That took inference from 17.9 ms to 6.0 ms with almost no loss in accuracy (97.22% to 96.62% on our test set at that stage), and it is why the listening core stays under 10%.

8. Deciding “was that Kunjika?”

The model gives a probability every 100 ms. Turning that stream of numbers into a clean yes/no decision is a small but important piece of logic.

8.1 A sliding window

Every **100 ms** (every 5 hops), the model looks at the **last 1 second**. The window slides along with time, so a word is seen several times as it passes: first partly, then fully, then partly again.

8.2 The four rules

Rule	Setting	Why
Smoothing	mean of the last 3 scores (0.3 s)	one lucky spike from noise cannot trigger the device
Threshold	0.80	the smoothed score must be at least this high; higher = fewer false triggers, more misses
Lockout	1,000 ms after a detection	the same word cannot trigger twice in a row
Re-arm	score must fall below 0.50 first	a long, slowly spoken word stays high for a while; without this it fired twice

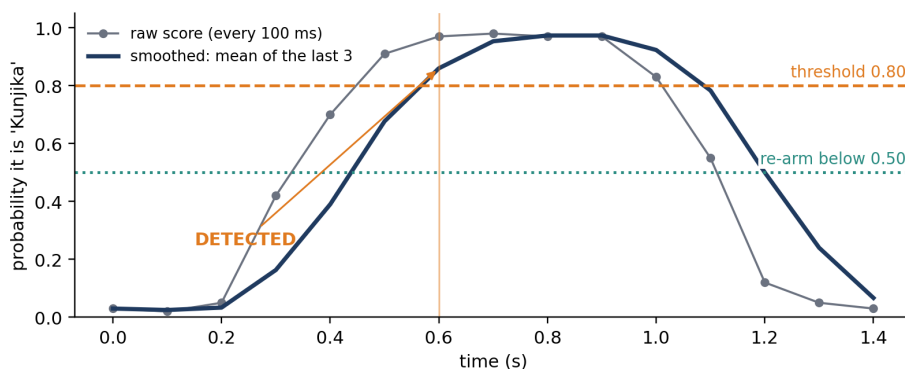


Figure 6. Real scores printed by our board during one detection. The raw score jumps quickly; the smoothed score crosses the threshold 0.3 s later; the device then waits for the score to drop below 0.50 before it can fire again.

8.3 The trade-off

Every keyword spotter balances two errors. A **miss** (false negative): you said Kunjika and nothing happened. A **false activation** (false positive): nobody said it, but the device woke up. Raising the threshold reduces false activations but causes more misses. We chose 0.80 from the training report, which lists both error rates at every threshold. We evaluated these rules in software with exactly the same logic the firmware uses, so the report predicts what the board will actually do (section 10.4).

In simple words

The board does not trust a single guess. It averages three guesses in a row, needs that average to be at least 80% sure, and after firing it waits until the word is clearly over before it can fire again.

9. Data: what the model learned from

A neural network is only as good as its examples. For a keyword spotter we need two kinds: **positives** (recordings of “Kunjika”) and **negatives** (everything that is *not* Kunjika: other words, conversation, noise, silence). Negatives matter just as much, because they are what teaches the model *not* to wake up.

9.1 The recording protocol

We wrote a recording protocol so that every speaker records the same set of conditions. Each speaker recorded all of them (a **crossed design**), so the model cannot learn “this voice always speaks loudly” or “this phone always means Kunjika”.

Condition	Count	Purpose
normal	30	the everyday way of saying it
fast / slow	10 each	different speaking speeds
quiet / loud	10 each	different volumes, and distance
lombard	10	said in a noisy place: people naturally raise and change their voice in noise
negatives (confusables)	20	kunji ka, kunji, kunj, kanjika, punjika, anjika: similar-sounding words
free speech	90 s	normal talking, as negative examples from the same voice, phone and room
silence	5 s	the room's own background sound

Why we did it this way

Negatives come from the **same speakers, phones and rooms** as the positives. Otherwise the model can take a shortcut: if every Kunjika was recorded on phone A and all the negatives came from the internet, it would learn to recognise phone A instead of the word. This kind of **shortcut learning** is a classic trap, and our own results later showed it happening (section 10.5).

9.2 What we collected

Source	Amount	Used as
9 team speakers (anuj, ayush, hitesh, lucky, ridhi, sanjay, subh, user1, vaibhav)	620 keyword clips; 591 kept	positives
Confusable words from the same speakers	130 clips	hard negatives
Free speech from the same speakers	11.7 minutes	negatives (same phones and rooms)
Held-out conversation (conv_test)	9.9 minutes	testing only: false activations per hour
Hindi podcast	38.2 min train / 18.8 min test	negatives / false-activation test
Room silence through the INMP441	150 s (5 takes of 30 s)	background noise for training; room tone for testing

Source	Amount	Used as
Speech Commands v0.02 (Google, open)	15,000 clips sampled	general word negatives from thousands of voices
Synthetic voices (Piper TTS)	7,154 clips	extra positives and hard negatives (names)
Friends saying the risky names	88 recordings, 3 people	testing only: do the names still trigger?

9.3 Preparing the data

The recordings arrived in every possible form: WAV, AAC and MP3; 44.1 kHz and 48 kHz; stereo and mono; folders named *Normal*, *quite* or *laoud*. Our script `prepare_dataset.py` converts everything to 16 kHz mono 16-bit WAV, maps the misspelled folder names to the right condition, cuts each keyword clip to the 1-second window that contains the most speech energy, and writes a **manifest** (a spreadsheet listing every file) plus a quality report.

The report found one important problem: many **slow** recordings were longer than one second, so the 1-second window could hold only 40 to 60% of the word. A clip labelled “Kunjika” that actually contains only “Kunj...” teaches the model that *part* of the word is enough, which is exactly how you get false triggers on “kunji”. So we drop any keyword clip with less than 70% of the word inside the window: **29 clips removed, 591 kept**.

9.4 Splitting by speaker

We split the data **by speaker, never by clip**: 6 speakers for training, 1 for validation (user1), and 2 for testing (subh and lucky). The test speakers' voices are never heard during training. If we split by clip instead, clips from the same person would appear on both sides, and the test would measure “does it recognise voices it already knows?” rather than “does it work for new people?”.

9.5 Synthetic voices (TTS)

Text-to-speech (TTS) software reads text aloud in a synthetic voice. We used **Piper**, an open-source TTS engine, with **129 voices**: 3 Hindi voices (pratham, priyamvada, rohan), 6 single-speaker English voices, and 120 voices from a multi-speaker English model. We generated “Kunjika” in several spellings (the Hindi voices got the word written in Devanagari script, so they pronounce it naturally), and 32 Indian names that sound similar to it, both alone and inside short sentences such as “mera naam Ranjita hai”. In total: 708 synthetic Kunjikas, 4,704 name clips and 1,742 sentence windows. About 15% of the voices (1,105 clips) were held out entirely, for testing.

What TTS taught us

Synthetic names helped, but they did not fully transfer to real people. On held-out synthetic voices, “Kritika” triggered the model 4% of the time; our friends saying “Kritika” triggered it 44% of the time. This is why we record real voices for testing and never trust synthetic results alone.

10. Training the model

Training runs in **Google Colab** (free cloud notebooks) with **TensorFlow/Keras**. The script *train_kunjika.py* loads the prepared data, builds the DS-CNN, trains it, applies quantization-aware training, converts it to an int8 TensorFlow Lite file, and then evaluates it exactly as the board would. One training run takes 30 to 45 minutes.

10.1 Augmentation: making the data harder

Augmentation means changing each training example a little, randomly, every time the model sees it. The model then learns what stays the same (the word) and ignores what changes (volume, noise, timing). Ours:

Augmentation	Setting	What it teaches
Time shift	± 100 ms	the word can be anywhere in the window (live audio slides past the model)
Gain	−20 dB to +6 dB	quiet and far-away speakers are still speakers
Background noise	80% of clips, 5–20 dB SNR	ignore fans, traffic and chatter behind the voice
Noise-only negatives	10% of negatives	pure noise is never the keyword
Room reverb	50% of clips, RT60 0.15–0.7 s	the echo of a real room does not change the word

Why time shift mattered so much

In our first prototype the test accuracy was excellent, but the live board missed words. The training clips all had the word neatly centred, while in live audio the 1-second window slides past the word, so the model mostly sees it off-centre. Shifting clips randomly by up to 100 ms fixed it. A good score on a test set does not guarantee good behaviour live.

10.2 What goes into each training batch

Each batch of 64 examples is drawn from seven sources in fixed proportions (model v3), so rare but important examples are guaranteed a share:

Source	Share	Source	Share
Real Kunjika	22%	Speech Commands words	18%
Synthetic Kunjika	8%	Same-phone speech and silence	16%
Real confusables	4%	Podcast and room tone	20%
Synthetic names	12%		

10.3 How we measure accuracy

- **Hit rate** (true-positive rate, recall): of all real “Kunjika”s from unseen speakers, what share was detected.
- **False activations per hour**: how many times the device would wake up during an hour of audio in which nobody says the keyword. This is the number users actually feel.

- **Confusable rate:** what share of similar-sounding words (kunji, kanjika...) fired.

10.4 Evaluating exactly like the board

The training script replays test audio through **exactly the firmware's logic**: features every 20 ms, inference every 100 ms, mean of 3, threshold, 1 s lockout, re-arm below 0.5. Each keyword clip is played with room tone before and after it, as the microphone would hear it. We checked this software replay against the firmware's own C code: same number of inferences, scores within 0.0005, and identical detections. The threshold table the script prints is therefore what the board will do.

10.5 Three versions and what each taught us

At threshold 0.80	v1	v2	v3 (current)
Hit rate, unseen speakers	80.4%	64.2%	76.4% (88% on normal speech)
Confusables that fired	67.5%	57.5%	45%
Podcast false activations	0 / hour	6.4 / hour	0 / hour (at every threshold)
Test speakers' free speech	4 in 3 min	3 in 3 min	2 in 3 min
Held-out conversation	not measured	not measured	3 in 9.9 min

- **v1** worked, but fired on confusables and on phone-recorded conversation. The model had partly learned “phone-recorded speech sounds like Kunjika”: the shortcut from section 9.1.
- **v2** forced 15% of every batch to be confusables. It backfired: some confusables (“kunji ka”, “kanjika”) are acoustically almost identical to the keyword, and with only 70 of them the model became hesitant about everything Kunjika-like, including real Kunjikas. We kept v1 and learned to balance hard negatives carefully.
- **v3** added synthetic voices and names in moderate shares. It is the best so far: zero podcast false activations at every threshold, fewer confusables, and 88 to 95% detection on normal, loud and noisy-place speech.

Condition (v3, threshold 0.8)	Hit rate	Name (real voices, 0.8)	Fired
normal	88%	Kritika	44%
loud	95%	Mallika / Ankita / Ambika	33%
lombard (noisy place)	95%	Sangeeta	29%
fast	75%	Kanika	11%
slow	33%	Deepika / Nikita / Monika / Radhika	0%
quiet	21%		

In simple words

The model is good at normal, loud and noisy-room speech, and it never woke up during 18.8 minutes of Hindi podcast. Its weak spots are whispered and very slow speech, and a few names that end like Kunjika. Chapter 16 lists how we are fixing those.

11. The firmware on the ESP32-S3

The board's software is written in C and C++ with **ESP-IDF** (Espressif's open-source development framework, version 6.1). ESP-IDF includes **FreeRTOS**, a small operating system that runs several **tasks** (independent loops of code) at once and decides which one runs when, based on priority.

11.1 Tasks and cores

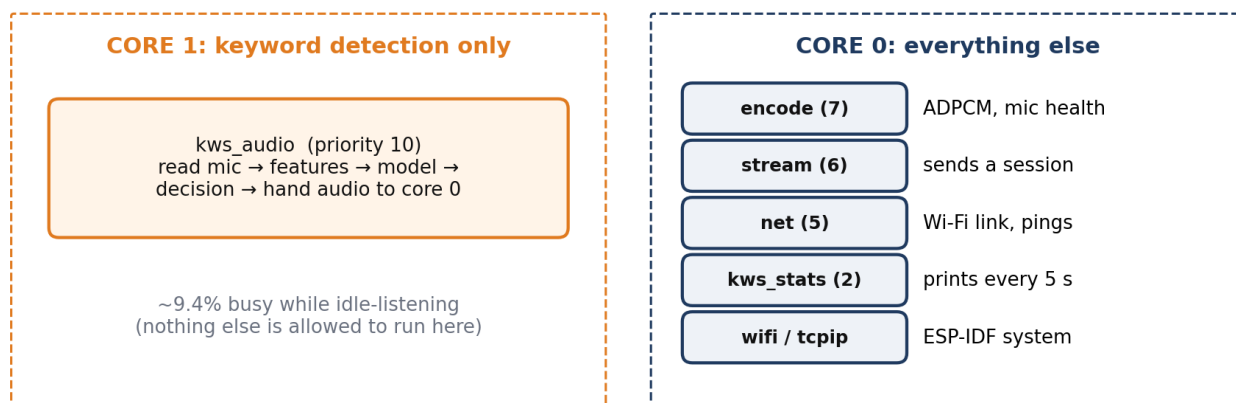


Figure 7. Which task runs on which core. The detection core runs a single task.

Task	Core	Priority	Job
kws_audio	1	10	reads the microphone, computes features, runs the model, decides, hands audio to core 0
encode	0	7	compresses every 20 ms hop with ADPCM into the 1.6 s history; checks microphone health
stream	0	6	on detection: sends the pre-roll and the live audio; ends the session
net	0	5	keeps the Wi-Fi and TCP connection alive; answers clock sync; watchdog
kws_stats	0	2	prints CPU, RAM, network and microphone statistics every 5 seconds
wifi, tcpip	0	system	ESP-IDF's own Wi-Fi driver and TCP/IP stack (lwIP), pinned to core 0 by us

Why we did it this way

The problem statement measures CPU *while idly listening*. Putting listening alone on core 1 means nothing else can steal its time or make its timing jittery. When we noticed the network stack running on core 1 during streaming (core 1 rose to 12–13%), we pinned it to core 0, and core 1 dropped back to about 10%.

11.2 The listening path (core 1)

The I2S hardware fills a buffer through DMA; the audio task wakes up every 20 ms with 320 fresh samples. It converts them to 16-bit, pushes them into the feature engine (*kws_engine*), which keeps the last 512 samples and a rolling 49-column feature buffer, and every 5th hop it runs the model. If the decision rules fire, it lights the LED and tells the streaming side. It

also hands every hop to core 0 through a small **staging ring** of 6 hops.

11.3 The streaming path (core 0)

Compression. The encoder task compresses every hop with **IMA-ADPCM**, which stores each sample as the *change* from a prediction, in 4 bits instead of 16. That is 4 times smaller, costs almost no CPU, and still sounds essentially identical (about 49 dB signal-to-noise ratio in our tests). The encoder also remembers its internal state at the start of every hop, so a session can start decoding at any hop.

History. The last **80 hops (1.6 s)** of compressed audio are kept in a ring buffer: 12.5 KB instead of the 51 KB it would take uncompressed.

A session. On detection, the stream task starts **1 second in the past** (50 hops), so the server receives the whole keyword and everything after it, even though detection comes a little after the word ends. Frames are packed into network writes of up to 1,400 bytes, so the 1-second pre-roll goes out in about 8 packets instead of 50. Live audio then follows hop by hop.

Session rule	Setting
Start	1 s before the detection (the pre-roll)
End	0.8 s of silence, but never sooner than 1 s after detection
Maximum	6 s after detection
“Silence”	a hop whose energy is less than 10 dB above the room's noise floor
Noise floor	20th percentile of the last 1.6 s of hop energies (ignores digital-silence dropouts)
Keyword end	the last loud hop before detection, if at least 2 quiet hops follow it

The **keyword end** estimate is sent to the server, which uses it both to measure latency and to cut the keyword off before transcription. If the speaker ran straight on into the command with no pause, the end cannot be measured, and the board says so honestly instead of guessing.

11.4 Networking

- **Wi-Fi station** mode, with power saving switched off (power saving delays incoming packets by up to one router beacon, about 100 ms).
- **Several networks:** `net_config.h` can list several networks, each with the server IP to use on it. The board tries them in order. If joining fails, it prints the reason in plain words (network not found, wrong password, and so on). It accepts WPA2 and WPA3.
- **An always-open TCP connection** to the server, opened at boot, so a detection never waits for a connection handshake. **TCP_NODELAY** is set, so small packets go out at once instead of being held back and batched.
- **Pings** every 2 s to measure round-trip time, and immediate answers to the server's **clock-sync** requests (chapter 12).
- **A silence watchdog:** the server sends a message every second, so if nothing arrives for 5 s the board declares the connection dead and reconnects. This fixed a real failure, a “half-open” connection (section 15).

- **A 2-second send limit:** a send can never hang forever on a dead link.

11.5 Reading the stats line

Every 5 seconds the board prints a block like this. Knowing what each field means is the fastest way to understand what the board is doing:

```
[ 45 s] CPU core1 (kws) 9.4% core0 3.1% avg 6.3% | kws work 9.1%
        50 inferences, max score 0.01, 0 detections | mic -30.8 dBFS
        | dropped audio 0, short reads 0
        net: 'MyHomeWiFi', round trip 13.0 ms (best 7.3 ms),
        reconnects 0
        heap (internal RAM): 89 KB in use, peak 102 KB, of 329 KB
        | free 239 KB
```

(The board prints each block on four long lines; they are wrapped here to fit the page.)

Field	Meaning
CPU core1 / core0 / avg	share of time each core was busy in the last 5 s; avg is the whole chip
kws work	time spent inside our listening code itself (the rest of core 1 is interrupts and DMA)
inferences, max score, detections	about 50 model runs per 5 s; the highest score seen; how many detections
mic	average microphone level; −30 to −45 dBFS in a quiet room; −120 means no signal at all
dropped audio, short reads	must stay 0: any other value means audio was lost
net	the network in use, current and best round-trip time, how many reconnections so far
heap	RAM allocated while running: now, the highest ever (peak), total available, and free
MIC FAULT (only if present)	share of audio that was digital silence or bit-pattern garbage: check the wiring
stack headroom (once, at 30 s)	unused stack bytes per task: used to size each task's stack from data

11.6 The microphone health check

The encoder task checks every hop for the two ways a badly connected INMP441 fails.

Digital silence: all 320 samples exactly zero; real sound is never exactly zero. **Bit-pattern garbage:** at least 90% of samples are 0 or of the form $2^k - 1$ (0, 1, 3, 7, 15 ... 511), the signature of a floating data wire. We first used a looser test, “no negative samples in a hop”, but the microphone's slow drift can keep a genuine 20 ms hop on one side of zero, so we tightened it; the new check raises no false alarms on real audio, even with strong drift.

11.7 The memory diet: from ~383 KB to ~243 KB

When we first added Wi-Fi and streaming, the whole firmware needed about 383 KB of internal RAM at its peak, far over the 256 KB limit. We measured where the memory went with *idf.py size* (static memory, per component) and the board's own heap statistics, then cut step by step:

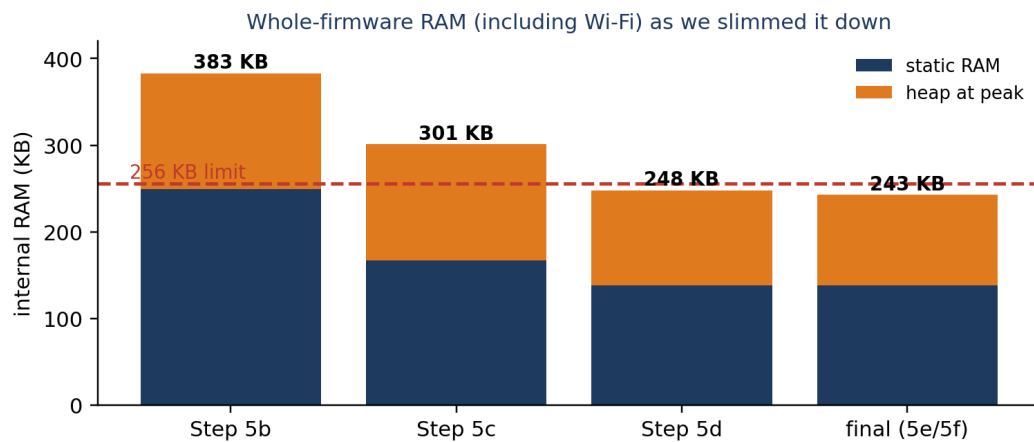


Figure 8. Total internal RAM (static + peak heap) of the complete firmware, including Wi-Fi.

Change	Saved
Audio history stored compressed (ADPCM), and 1.6 s instead of 2 s	~50 KB static
Wi-Fi, TCP/IP, FreeRTOS and heap code run from flash instead of RAM; flash driver from the chip's ROM	~30 KB static
Removed a copy of the model in RAM (measured: it saved no CPU)	~17 KB static
Model arena 32 KB → 18 KB (16.9 KB is actually used)	14 KB static
Task stacks sized from measured headroom	~8 KB heap
Pre-roll sent as ~8 packed packets instead of 50 small ones	~20 KB peak heap
Fewer Wi-Fi buffers (sized for Mbit/s by default; we need 71 kbit/s); TCP send buffer 2.9 KB	peak heap, and bounded during network stalls

Final figure: **137.7 KB static + 105 KB peak heap ≈ 243 KB** for the complete firmware including Wi-Fi, and about 227 KB while idle. The keyword-spotting part alone is small: the model's working memory is 16.9 KB, and the feature buffers a few KB more.

One honest caveat on RAM

The 137.7 KB static figure comes from the last *idf.py size* before the final buffer-setting tweak. That tweak changed network buffer counts, not static variables, so the figure should hold, but it is worth running *idf.py size* once more before quoting it. The peak heap of 105 KB was measured over a full session run in a quiet room.

12. How the board and the server talk

The board and the server exchange small messages over one TCP connection. Every message, called a **frame**, has the same simple shape: 1 byte saying what kind of message it is, 2 bytes giving the length of what follows, then the payload.

Frame	Direction	Contents and purpose
HELLO	board → server	protocol version, sample rate, device name
PING / PONG	board ↔ server	board's timestamp, echoed back: round-trip time
SYNC / SYNC_REPLY	server ↔ board	server's timestamp, echoed with the board's: clock synchronisation
START	board → server	session id, detection time, keyword-end time, time of the first audio sample
AUDIO	board → server	one 20 ms hop: sequence number, ADPCM state, 160 bytes of audio
END	board → server	why the session ended (silence, 6 s limit, connection), number of samples
FIRST_ACK	server → board	"I received your first audio", sent the instant it arrives
TEXT	server → board	the transcript (empty when there was nothing to transcribe)

12.1 Clock synchronisation: measuring latency honestly

To measure "keyword end → server receiving audio", we need the keyword-end time (known by the board) and the arrival time (known by the server) on the **same clock**. But the board's clock counts from its own power-on, and the laptop's clock is completely different. So we synchronise them:

1. Every second, the server sends SYNC with its current time T_s .
2. The board replies immediately with T_s and its own time T_b .
3. The reply arrives at server time T_r . The round trip took $T_r - T_s$, so the board most likely read its clock halfway through: $\text{offset} = T_b - (T_s + \text{round-trip} / 2)$.
4. We keep the exchange with the **shortest** round trip out of the last 20, because it has the least uncertainty. The error can be at most half that round trip: typically ± 3 ms.

A worked example. The server sends at 1,000,000 μs ; the reply arrives at 1,006,000 μs (a 6 ms round trip) carrying board time 5,003,100 μs . $\text{Offset} = 5,003,100 - 1,003,000 = 4,000,100 \mu\text{s}$. A keyword that the board says ended at 5,500,000 μs therefore ended at server time 1,499,900 μs ; if its first audio arrived at 1,570,000 μs , the latency was 70.1 ms.

If the clock uncertainty ever exceeds ± 20 ms (for example, the laptop went to sleep), the server refuses to report a latency for that session rather than log a misleading number.

In simple words

The board and the laptop each have their own clock. By bouncing timestamps back and forth every second, the server works out how far apart the two clocks are, to within about 3 ms. That lets it convert "the keyword ended at board time X" into laptop time, and measure the delay precisely.

13. The server: speech recognition

The server, `kws_server.py`, is a Python program that plays the role of “the cloud”. It uses **asyncio**, Python's way of handling many network events in a single thread without waiting on any one of them.

13.1 What it does for every session

1. Receives the frames and prints the latency the **instant** the first audio arrives: `>> session 3: ASR receiving audio - 71 ms after the keyword ended.`
2. When the session ends: decodes the ADPCM, saves a WAV file, and writes a row to `sessions.csv` (latency, its detection and network parts, audio length, data rate, transcript).
3. Hands the audio to Whisper **in the background**, then sends the transcript back to the board.

13.2 Whisper

Whisper is an open-source speech-recognition model released by OpenAI. It understands many languages, including Hindi and English. We use **faster-whisper**, an open-source re-implementation that runs the same model on the **CTranslate2** engine with int8 maths, which is several times faster on a CPU. We use the **small** model (about 244 million weights).

Whisper always processes a 30-second window, padding short clips with silence. On our laptop's CPU one transcription takes 5 to 10 seconds. That time is not part of the latency metric; on a real cloud GPU it would be a fraction of a second. An experimental `--fast` option asks Whisper for a 10-second window instead, and falls back automatically if the installed version does not support it.

13.3 Only the command, never the keyword

The board has already confirmed the keyword, so the server cuts the audio at the keyword end (or 0.25 s before detection when the speaker ran straight on) and sends *only the command* to Whisper. “Kunjika,” is then written in front. This helps in two ways: the Hindi-sounding keyword no longer confuses Whisper's language detection, and the transcript always spells the keyword correctly. Any leftover tail of the keyword (Whisper sometimes writes “jeeg” for “-jika”) is removed by comparing **sounds**, not spellings, while real words such as “kya”, “kaun” and “kunji” are kept. We tested this on 24 examples, all correct.

13.4 English, Hindi and Hinglish

- **One pass when the language is clear.** Whisper detects the language and transcribes it in the same pass.
- **Two passes when it is not.** If Whisper is less than 85% sure, or gives Hindi at least 15% probability even when English wins, the command is transcribed both as English and as Hindi, and the version Whisper is more confident in is kept. When Whisper decides a Hindi sentence is English, it tends to *translate* it, and this check stops that.
- **Only English or Hindi are accepted.** Whisper sometimes labels noise as Korean or Georgian; we never print those scripts.

- **Hinglish output.** Hindi is transcribed in Devanagari (the accurate path) and then converted to Roman letters by our own transliterator, *hinglish.py*. Its key rule is **schwa deletion**: Hindi spelling has a silent “a” that people drop in speech: the Hindi word for “does” is spelled with letters that read *ka-ra-ta*, but everyone says and types *karta*. Common English words that Whisper writes in Devanagari (such as the words for “light” and “on”) are mapped back to their English spelling, *light* and *on*.

13.5 Guards against Whisper's bad habits

Habit	Guard
Hearing “you” or “Thank you.” in silence	Whisper's own no-speech probability above 0.6, or a known silence phrase with low confidence → just “Kunjika”
Copying example text from its prompt (“pankha band karo” that nobody said)	no example sentences in any prompt
Looping (“aap aap aap aap”)	no repeated 3-word sequence; at most 48 tokens; no temperature retries
Guessing when unsure	average confidence below -1.0 (log-probability) → “(not understood)”
Half a Devanagari character at a token boundary (shows as a broken-character symbol)	removed

13.6 Keeping the network side fast

Transcription runs in a **separate process** at below-normal priority, using about half the CPU threads. More importantly, the connection's reader **never waits** for a transcription to finish. Our first version did wait, and the board's next session sat unread in the network buffer for 4 seconds (section 15 tells that story).

13.7 Optional: a Hindi specialist model

--*hindi-model* can load **Whisper Hindi Small** from IIT Madras (vasista22/whisper-hindi-small), a version of Whisper fine-tuned on Hindi, whose compute was funded by MeitY's Bhashini mission. It is released under the open Apache-2.0 licence and reports a word error rate of about 9% on the FLEURS Hindi test set. It must be converted once into CTranslate2 format. It is not set up yet; it is the main planned step for better Hindi transcripts.

14. Results: what we measured

These are the final numbers, with how each one was measured, so anyone on the team can defend them.

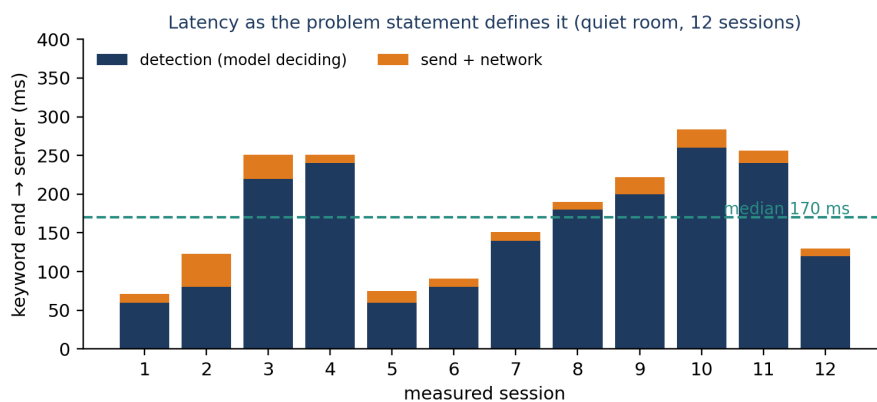


Figure 9. Latency of the 12 sessions measured in two quiet-room runs: mostly model decision time (navy), with only 10–43 ms for sending and the network (orange).

Metric	Result	How it was measured
Latency (keyword end → server receives audio)	median 170 ms; 90th percentile 255 ms; range 71–284 ms	server clock at the first AUDIO frame minus the board's keyword-end time converted with the synchronised clock (± 3 ms)
...detection part	median 160 ms	board: detection time minus keyword-end time
...send + network part	median 13 ms	the rest
One-breath sessions (detection → server)	10–59 ms	same method; the keyword end is not measurable in these
CPU, idle listening	detection core 9.3–9.6%; chip average 6.3%	FreeRTOS run-time statistics every 5 s, idle-task time subtracted
RAM, whole firmware	137.7 KB static + 105 KB peak heap $\approx$ 243 KB (idle $\sim$ 227 KB)	idf.py size for static; the board's heap statistics for the peak
Model	16,808 bytes int8; 16,892-byte arena; 6.04 ms per run; $\sim$ 0.55 M multiply-adds	file size; TFLM arena report; per-layer profiler
Features	1,960 of 1,960 values identical to TensorFlow	same audio through both, compared value by value
Audio data rate	71 kbit/s (256 kbit/s raw); 49 dB SNR after compression	bytes on the wire per second of audio; decoded vs original
Audio integrity	0 dropped audio, 0 short reads	board counters over every run
Accuracy (v3)	88% normal speech; 0 false activations in 18.8 min podcast; 3 in 9.9 min conversation	device-identical replay of held-out speakers and audio

15. Bugs we hit, and what they taught us

Every one of these cost us time. Knowing them will save yours, and several make good answers when judges ask “what was hard?”.

Symptom	Cause	Fix, and lesson
Microphone silent or producing garbage, on and off, for days	INMP441 header pins pushed in, never soldered	Solder them. The firmware now detects both failure patterns. Check hardware before software.
A changed file had no effect after building	zip timestamps (UTC) looked older than the last build (IST)	Refresh the timestamps of *.c, *.cc, *.h and CMakeLists.txt after every unzip.
Recorder output was scrambled	the PC listened at 921,600 baud, the board sent at 115,200	Both sides must use the same speed.
Features almost, but not exactly, right	symmetric instead of periodic Hann window	Compare every value against the reference, not just the final accuracy.
Great test accuracy, poor live detection	test words were centred; live words slide past the window	Time-shift augmentation. Test the way the device is actually used.
One long word fired twice	the score stayed high after the lockout ended	Re-arm only after the score drops below 0.5.
Model v2 detected less	too many near-identical confusables in every batch	Balance hard negatives; some “confusables” are the keyword with a pause in it.
Synthetic names rejected, real names fired	TTS voices do not fully transfer to real people	Always test on real voices; record real names for training.
Transcript showed a command nobody said	Whisper copied the example sentences in its prompt	No example commands in prompts.
Board stuck forever after a Wi-Fi hiccup	half-open TCP connection: no “goodbye” ever arrived	5 s silence watchdog and a 2 s send limit.
4-second stall before the next session	the server's reader waited for the previous transcription	Transcribe in the background. Our first diagnosis (CPU starvation) was wrong: reproduce before fixing.
“Kunjika, you” in every transcript	Whisper hears “you” in silence (the mic was faulty)	Use Whisper's no-speech probability; fix the microphone.
Board never joined a phone hotspot	hotspot used WPA3 only; another time the password was wrong	The board now prints the reason (wrong password, not found...) and accepts WPA3.
Build error on generated headers	the generator wrote 60f instead of 60.0f	Compile-test generated code against the real headers, not old ones.

16. What is still weak, and the plan

Being clear about weaknesses is part of good engineering, and judges trust teams that measured their own. If we are selected, the month before the grand finale goes into these, in this order:

Weakness	Evidence	Plan
False activations in conversation	3 in 9.9 min of held-out conversation (v3)	1–2 hours of same-phone conversation for training; hard-negative mining (retrain on the moments the model almost fired)
Some names trigger it	Kritika 44%, Mallika / Ankita / Ambika 33% (real voices)	real recordings of these names from all team speakers, for training
“Kunjika” + command in one breath	sometimes missed; works with a 0.5 s pause	continuation augmentation: splice real speech right after the keyword in training; a test set of real one-breath commands
Quiet and slow speech	21% and 33% hit rate (v3)	more quiet and slow recordings; loudness augmentation
Detection delay is most of the latency	median 160 ms of the 170 ms	a more confident model (all of the above); tune smoothing
CPU margin	9.3–9.6% vs the 10% limit	optional: run the model every 120 ms instead of 100 ms (about 1% less CPU)
Hindi transcripts	about 2 in 3 good with Whisper-small	the IIT Madras Hindi model (section 13.7)
Transcription time on a laptop	5–10 s	a real cloud server with a GPU; or the --fast window

17. Running everything yourself

17.1 The server (laptop)

```
cd kws_step5\server
pip install numpy faster-whisper          (first time only)
python kws_server.py                      (listens on port 7000, auto English/Hindi)
python kws_server.py --fast                (experimental 10 s window)
python kws_server.py --model base          (faster, weaker for Hindi)
```

Allow Python through the Windows firewall on **private** networks. Keep the laptop plugged in, set its Wi-Fi adapter to Maximum Performance, and do not let it sleep during a test.

17.2 The board

Edit `kws_step5\main\net_config.h` with your network name, password and the laptop's IP (from `ipconfig`), then in the ESP-IDF PowerShell:

```
cd kws_step5
Get-ChildItem main\* -Include *.c,*.cc,*.h,CMakeLists.txt |
  ForEach-Object { $_.LastWriteTime = Get-Date }      (one command, split here to fit)
idf.py build
idf.py -p COM6 flash monitor                          (exit the monitor with Ctrl+]
idf.py set-target esp32s3                             (only when sdkconfig.defaults changed)
idf.py size                                             (static RAM report)
```

17.3 When something goes wrong

What you see	Likely cause	What to do
mic -120.0 dBFS, or MIC FAULT	microphone wiring	check VDD, GND and SD first; resolder
net: cannot join '...': network not found	wrong name, a 5 GHz-only network, or out of range	exact name; 2.4 GHz; move closer
net: cannot join '...': wrong password	password	check WIFI_PASS (case-sensitive)
net: cannot reach server	server not running, wrong IP, firewall	start the server; ipconfig; allow Python
COM6 not found	cable, USB port, or a new COM number	reseat the cable; list the ports; use a data cable
a change had no effect after unzip	old timestamps	run the timestamp line above
transcript "(not understood)"	unclear audio	speak closer, then pause after the command

17.4 Where everything lives

File	What it does
main/main.cc	starts everything; audio task; statistics task; LED
main/kws_engine.cc/.h	features and model: the listening engine

File	What it does
main/kws_model.h, kws_mel.h, kws_config.h, kws_ops.h	generated from the trained model by tools/gen_step4.py
main/stream.c/.h	encoder task, compressed history, sessions, microphone health
main/net.c/.h, net_config.h	Wi-Fi, networks list, TCP connection, watchdog, clock-sync replies
main/adpcm.c/.h	IMA-ADPCM encoder (bit-identical to the server's decoder)
sdkconfig.defaults	build settings, including all the RAM savings
server/kws_server.py	the server: sessions, latency, Whisper, CSV
server/hinglish.py	Devanagari → Hinglish transliteration
server/fake_board.py	a pretend board, for testing the server without hardware
prepare_dataset.py, gen_tts.py, train_kunjika.py	data preparation, synthetic voices, training (in Colab)

18. Glossary

Every important term in this document, in plain words, in alphabetical order.

ADPCM	Adaptive Differential Pulse-Code Modulation. Instead of storing each sample, it stores the small change from a prediction, using 4 bits instead of 16. Four times smaller, very cheap to compute, almost no audible loss. We use the IMA variant.
Arena (tensor arena)	The block of RAM that TensorFlow Lite Micro uses for the data flowing between the model's layers. Ours uses 16.9 KB.
ASR	Automatic Speech Recognition: software that turns speech into text. Whisper is our ASR.
Augmentation	Randomly altering training examples (shift, volume, noise, echo) so the model learns what matters and ignores what does not.
Average pooling	Replacing a whole grid of numbers with its average. Our model uses it to summarise each channel over the full second of sound.
Batch	The group of examples (64 for us) the model learns from in one training step.
Batch normalisation	A layer that rescales values inside the network to a steady range, which makes training stable.
Beam search	Whisper keeps several candidate sentences (5 for us) while decoding and picks the most likely one.
Channel	One pattern detector in a convolution layer. Our layers have 32 channels each.
Clock synchronisation	Working out the difference between the board's clock and the server's clock by exchanging timestamps, so that times from both can be compared (chapter 12).
Confusable	A word that sounds similar to the keyword (kunji, kanjika). Used as a hard negative.
Convolution	Sliding a small pattern detector over a picture and measuring how well each spot matches.
Core	One independent processor inside the chip. The ESP32-S3 has two; we give core 1 only the listening task.
CPU utilisation	The share of time a processor is busy. We measure it per core from FreeRTOS run-time statistics.
CTranslate2	An open-source engine that runs Transformer models such as Whisper quickly on CPUs; used by faster-whisper.
dB / dBFS	Decibels: a logarithmic scale for loudness. dBFS is relative to the loudest possible value (0 dBFS); -6 dB halves the amplitude.
Dense layer	A layer in which every input is connected to every output; our last layer turns 32 numbers into 2 scores.
Depthwise separable convolution	A convolution split into a per-channel spatial filter plus a 1×1 channel mixer: about 7 times cheaper for our sizes.

DMA	Direct Memory Access: hardware that moves data (here, microphone samples) into memory without using the CPU.
Dropout	During training only, randomly hiding some values so the model cannot depend on any single one.
DS-CNN	Depthwise Separable Convolutional Neural Network: the standard small architecture for keyword spotting, and ours.
Edge	The device near the user (our ESP32-S3), as opposed to the cloud.
ESP-IDF	Espressif's open-source development framework for ESP32 chips; includes FreeRTOS, Wi-Fi and lwIP. We use version 6.1.
ESP-NN	Espressif's library of optimised neural-network layers that use the ESP32-S3's vector instructions.
False activation	The device wakes up although nobody said the keyword (a false positive). Measured per hour of audio.
faster-whisper	An open-source, faster re-implementation of Whisper built on CTranslate2.
FFT	Fast Fourier Transform: splits a chunk of sound into how much energy it has at each frequency.
Flash	Non-volatile storage on the board (16 MB) that holds the program and the model; it keeps its contents without power.
Float32 / int8	32-bit decimal numbers versus 8-bit whole numbers (−128 to 127). int8 is 4× smaller and faster on a microcontroller.
Frame / hop	A frame is the 512-sample (32 ms) window the features are computed from; the hop is the 320-sample (20 ms) step between frames. Everything in the system ticks once per hop.
FreeRTOS	A small open-source real-time operating system that runs several tasks and schedules them by priority.
Hann window	A smooth fade-in and fade-out applied to a frame before the FFT, to avoid fake frequencies from sharp edges.
Hard negative	A negative example that is easy to confuse with the keyword; the most useful and most dangerous training data.
Heap	RAM handed out while the program runs (Wi-Fi buffers, task stacks). We report its peak.
Hinglish	Hindi written in Roman letters, as people type it (“light on karo”).
Hit rate	The share of real keywords that are detected (true-positive rate, recall).
I2S	Inter-IC Sound: a three-wire standard for sending digital audio between chips (clock, word select, data).
Inference	Using a trained model to make a prediction. Ours runs every 100 ms and takes 6 ms.
INMP441	Our digital MEMS microphone; it sends 24-bit audio over I2S.
IRAM / DRAM	Parts of the ESP32's internal RAM for code (instruction RAM) and data (data RAM). Code that runs from RAM is fast but uses up the precious 512 KB, so we moved much of it to flash.

Keyword spotting (KWS)	Continuously listening for one specific word; the “wake word” detector.
Latency	Delay. In the problem statement: the time from the keyword ending to the ASR server receiving the audio.
Lockout / re-arm	After a detection the device ignores the keyword for 1 s, and fires again only after the score drops below 0.5.
Log-mel spectrogram	A picture of sound: time left to right, 40 pitch bands bottom to top, log of energy as brightness.
Logits	The raw scores a network outputs before they are turned into probabilities.
Lombard effect	People automatically speak louder and change their voice in noisy places.
lwIP	The lightweight open-source TCP/IP network stack inside ESP-IDF.
Mel scale	A pitch scale that matches human hearing: fine at low frequencies, coarse at high ones.
MEMS	Micro-Electro-Mechanical System: a microscopic moving part on a chip, here a microphone membrane.
Microcontroller (MCU)	A small, low-power computer on a single chip, such as the ESP32-S3.
Noise floor	The level of the room's background sound; our board estimates it as the 20th percentile of recent hop energies.
Nyquist limit	With N samples per second you can capture frequencies up to N/2. 16 kHz sampling covers speech up to 8 kHz.
Percentile	The value below which that percentage of the data falls; the 20th percentile is a robust “typical quiet” level.
Pre-roll	The 1 second of audio before a detection, kept in memory and sent first, so nothing between the word and the detection is lost.
PSRAM	Extra, slower RAM outside the chip (8 MB on our board). Deliberately switched off so our RAM figure is honest.
QAT	Quantization-Aware Training: training while simulating int8 rounding, so the model stays accurate after quantization.
Quantization	Storing numbers with fewer bits (float32 → int8) using a scale and zero point.
RAM (static vs heap)	Static RAM is reserved when the program is built (variables, buffers); heap is taken while running. Total = static + peak heap.
ReLU	Rectified Linear Unit: replaces negative numbers with zero; the simple bend that lets a network learn complex shapes.
Reverb / RT60	Echo in a room; RT60 is the time for the echo to fade by 60 dB (0.15–0.7 s in our augmentation).
Ring buffer	A fixed-size buffer that wraps around, always holding the most recent data (our last 1.6 s of audio).
Round-trip time (RTT)	Time for a message to reach the other side and a reply to come back.

Sample / sample rate	One measurement of the sound wave; we take 16,000 per second.
Schwa deletion	In spoken Hindi the inherent “a” of a consonant is often dropped (written ka-ra-ta, spoken karta); our transliterator applies this rule.
Session	One streaming episode: from a detection until the speaker stops (or 6 s).
Sigmoid	The function $1 / (1 + e^{-x})$ that turns any number into a probability between 0 and 1.
SNR	Signal-to-Noise Ratio, in dB: how much louder the wanted sound is than the noise.
Softmax	Turns several scores into probabilities that add to 1; for two classes it equals a sigmoid of their difference.
Speaker split	Dividing data so that test speakers are never heard in training: a fair test of new voices.
Stack (task stack)	Each task's own scratch memory for its function calls; we sized ours from measured headroom.
Task	An independent loop of code that FreeRTOS schedules; our firmware has five of its own.
TCP	Transmission Control Protocol: a reliable, ordered connection between two computers over a network.
TCP_NODELAY	A socket setting that sends small packets immediately instead of batching them (Nagle's algorithm).
TensorFlow Lite Micro (TFLM)	Google's open-source runtime for running neural networks on microcontrollers.
Threshold	The minimum smoothed score (0.80) needed to declare a detection.
TTS	Text-to-speech: software that reads text aloud in a synthetic voice. We used Piper to create training voices.
Watchdog (connection)	Our rule that 5 s without any message from the server means the connection is dead, so reconnect.
Whisper	OpenAI's open-source speech-recognition model; our ASR (the “small” size).
Word error rate (WER)	The share of words an ASR system gets wrong; lower is better.
WPA2 / WPA3	Wi-Fi security standards; WPA3 is the newer one. Our board accepts both.
Zero point / scale	The two numbers that convert int8 values back to real values: $\text{real} \approx \text{scale} \times (\text{int} - \text{zero point})$.